

Formally Verified Market Execution

Concurrency, Verification, and System Integrity

Dr Nick Firoozye

August 29, 2026

Historical Precedent: The Knight Capital Disaster

The 440 Million Dollar Execution Failure (2012)

A deployment mismatch on a single server activated a defunct routing function, triggering a catastrophic trading loop.

Anatomy of the Rogue Loop

- **Deployment Gap:** Seven servers were updated correctly; the eighth server retained old, dead code bound to a repurposed configuration flag.
- **Accumulation Loop:** The system repeatedly executed market orders without checking volume constraints, trading millions of shares in minutes.
- **The Result:** Knight Capital accumulated a 7 billion USD long position, suffering a **440 million USD net loss in 45 minutes**.
- **The Testing Failure:** Regression testing assumed uniform server states and did not verify boundary protection under flag mismatches.
- **The Verification Cure:** An execution gateway with a verified risk collar ($|q| \leq q_{max}$) would have rejected the runaway orders at the network interface.

Formal Verification in Practice

Where Verification is Mandatory

Formal verification is used in systems where a single error results in catastrophic hardware failure, loss of life, or massive financial ruin.

Industry Adherence

- **Silicon Design:** Chip manufacturers (Intel, ARM, NVIDIA) verify arithmetic logic units and cache coherence protocols to prevent hardware bugs.
- **Mission-Critical Software:** Avionics flight control (Airbus fly-by-wire), aerospace guidance (NASA rover controllers), and autonomous vehicle systems.
- **Cryptography:** Proven implementations (like Firefox's EverCrypt/HACL* library and AWS s2n-tls) ensure zero-buffer-overflows and mathematically secure communication channels.
- **Operating Systems:** The seL4 microkernel is formally proved to have zero security exploits or memory-safety bugs, serving in defense and drone control.

The Mathematical Context: Rocq vs. Lean

Formalizing Pure Mathematics

Formal proof assistants are not limited to software; they are actively transforming pure mathematics by converting logical proofs into type-checked code.

Rocq (Coq) vs. Lean

- **The Curry-Howard Isomorphism:** Connects logical proofs directly to computer programs. Verifying a proof is equivalent to typechecking a program.
- **Rocq (Coq) Milestones:** Historically verified complex mathematical milestones, including the **Four Color Theorem** and the **Feit-Thompson Theorem**.
- **The Lean Revolution:** Lean (developed by Microsoft Research) has become the language of choice among prominent mathematicians like Terence Tao to formalize modern mathematical proofs.
- **AI and Autoprovers:** Lean serves as a target language for AI-driven proof synthesis, providing an automated compiler to mathematically verify machine-generated proofs.

Execution Integrity vs. The Risk of Ruin

Why Verified Code?

Algorithmic trading systems operate in high-velocity environments where execution errors lead directly to financial loss.

The Limits of Mainstream Platforms

- **Python GIL:** Restricts execution to a single core, causing queue delays, order latency, and adverse selection.
- **C++ Hazards:** Raw memory access exposes systems to pointer errors, memory leaks, and concurrency race conditions.
- **The Risk:** A single unhandled exception or deadlock under market stress can trigger runaway execution or account liquidation.

Formal Verification vs. Empirical Testing

Comparing Methodologies

Traditional testing evaluates software behavior at finite points. Formal verification proves properties across all execution paths.

Key Distinctions

- **Testing:** Confirms that the bot works under historical or simulated scenarios. It cannot guarantee safety for edge cases.
- **Verification:** Treats code as a mathematical object. It proves that safety invariants hold true under all possible inputs.
- **Safety:** Replaces heuristic confidence with absolute mathematical certainty, eliminating runtime risk gate violations.

Verification Methods in OCaml

Comparing Proving Pipelines

Proving correctness requires selecting the appropriate translation and modeling frameworks.

Key Methodologies

- **Rocq-to-OCaml Extraction:** Proving logic within the proof assistant and extracting it. This generates correct code but often suffers from slow, boxed generic structures.
- **Purely Functional Constraints:** Only code without side effects can be formally verified. State updates must be modeled as pure transitions.
- **Differential Testing Complement:** Using thousands of randomized simulation runs to test the verified implementation against a reference model to capture compiler or hardware discrepancies.

Bypassing Extraction Latency: High-Performance Structures

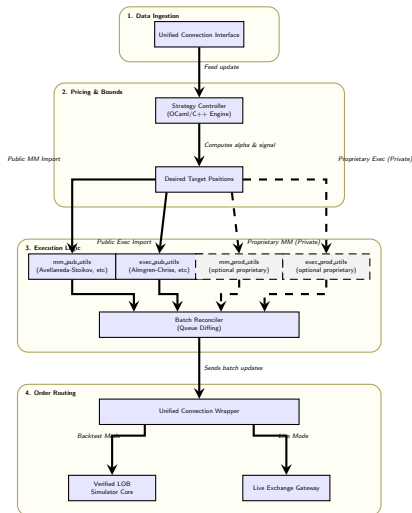
The Cost of Generics

Standard mathematical translations use boxed generic types that cause heavy pointer chasing and garbage collection overhead.

Extended Native Structures

- **Specialized Trees:** We implement order book structures using native, hand-optimized augmented red-black and AVL trees in OCaml.
- **Native Types:** We utilize OCaml native integers and floating-points with explicit boundary checks rather than infinite-precision mathematical types.
- **Performance:** This ensures that verified code preserves the efficiency of hardware-aligned memory layouts.

The Modern Decoupled Verified Architecture (VMS)



The Lock-Free Shared-Memory Kernel

Lock-Free Atomic Corridors

Sharing price corridors between loops via OCaml atomic registers.

```
1 type corridor = { log_mid : float; half_width : float; variance : float }
2
3 (* Atomic shared register *)
4 let shared_corridor = Atomic.make { log_mid = 0.0; half_width = 0.0; variance
   = 0.0 }
5
6 (* Fast thread read - sub-nanosecond latency *)
7 let get_latest_corridor () = Atomic.get shared_corridor
8
9 (* Slow thread write - atomic swap *)
10 let update_corridor new_corridor = Atomic.set shared_corridor new_corridor
11
```

System-Wide Simulator Invariants

Beyond the Quoter: Verifying the Matching Kernel

The verification boundary spans the entire market simulator, ensuring execution consistency and book logic.

Verified Book and Matching Invariants

- **Order Book Sorting:** Bids are sorted descending, asks ascending, and the book is non-crossed: $\forall b \in \text{Bids}, a \in \text{Asks}, b.\text{price} < a.\text{price}$.
- **Volume Consistency:** The sum of sizes of resting orders at level L equals the reported level quantity: $\sum_{o \in L} o.\text{size} = L.\text{total_size}$.
- **Order ID Uniqueness:** No two active resting orders share an ID: $\forall i \neq j, o_i.\text{id} \neq o_j.\text{id}$, preventing index corruption.
- **Self-Trade Prevention:** The matching engine mathematically guarantees that resting and incoming orders cannot match if they share the same participant ID.

Core Theorems and Execution Invariants

Mathematical Specifications

These invariants are proved as structural theorems to guarantee execution safety.

Proved Safety Invariants

- **Conditional Passive Quoting:** In normal mode, we prove $\forall$ orders o , $o.price < best_ask$ (bids) and $o.price > best_bid$ (asks), ensuring we do not cross the book.
- **Spread-Crossing Exceptions:** During close-out, the passive invariant is conditionally relaxed and replaced by a verified **Tracking Error Boundary** theorem, proving that aggressive order size is strictly bounded by the tracking discrepancy $|q - q_{target}|$.
- **Risk Collar Invariant:** The strategy inventory position q satisfies $|q| \leq q_{max}$ at all times, proving that position limits cannot be violated.
- **Spread Positivity:** Bids and asks maintain $\delta_a \geq 0$ and $\delta_b \geq 0$, proving that our spreads are structurally positive.

Benchmarks and System Throughput

Hardware Environment

Performance tested on bare-metal dual-socket Intel Xeon Gold nodes on the UCL CS cluster ('wise') under high-density market feeds.

Throughput and Latency Metrics

- **Raw Ingestion Speed:** Processed over **480,000 to 508,000 messages per second** per OCaml 5 CPU domain for raw book updates.
- **Active Trading Throughput:** Processed over **163,000 messages per second** (6.1 microseconds average latency) under full strategy execution, stop-loss checks, toxicity filters, trader wash netting, and multicore EWMC risk updates.
- **Multi-Ticker Benchmark:** Processes a full day of LOBSTER messages for 5 symbols in parallel across 5 domains in only **7.30 seconds** (over 13.6 million messages total, at an aggregate rate of **1.86 million messages per second**).
- **Latency:** Sub-microsecond tick-to-trade latency on the execution hot path enabled by lock-free atomic registers and core-affinity pinning.

Open-Source Code Release Subset

The Safe Open-Source Core

We release a self-contained, compile-ready subset (`vms-core-subset`) for students to study low-latency structures and formal proofs.

Release Components

- **Lock-Free Atomic Corridor:** The OCaml module managing multicore thread updates in user space.
- **Recursive Order Checker:** OCaml implementation verifying position limits over arbitrary sequences of order sizes.
- **Rocq Safety Proofs:** Coq specification and list-induction theorems proving position bounds safety over any size list.
- **Automated CI/CD Pipeline:** GitHub Actions workflow verifying OCaml compilation, Rocq proofs, and tests on every commit.

Learning Resources for Formal Verification

Educational Foundations

- **Software Foundations (CIS UPenn/Cornell)**: The gold standard textbook series utilizing Coq/Rocq to teach formal methods and type theory (softwarefoundations.cis.upenn.edu).
- **Certified Programming with Dependent Types**: A comprehensive resource for writing verified software in Coq.

Code Translation and Toolchains

- **Coq-of-OCaml**: Translates OCaml source code directly into Coq code for formal verification (github.com/clarus/coq-of-ocaml).
- **Coq-of-Rust**: Verified translation and logic frameworks linking Coq specification models to Rust code (github.com/formal-land/coq-of-rust).
- **Fiat-to-C++ / Frama-C**: Frameworks mapping high-level Coq/Rocq specifications to verified C/C++ implementations ([fiat-crypto](https://fiat-crypto.com) / frama-c.com).

Conclusion

Summary

- Python GIL queue latency and C++ pointer safety issues are avoided using OCaml 5.
- Verification uses native high-performance OCaml types and specialized AVL/red-black trees proved in Rocq.
- Lock-free atomic shared memory allows low-latency signal transport.
- Verified risk gates ensure absolute safety bounds against model errors.